Fronthaul Load Management for Open RAN

IQ Compression by Matrix Decomposition and Deadline-Feasible, Uncertainty-Aware Budget Allocation on a Shared 7-2x Link

Mayank Yadav Harshvardhan Choudhary

B.Tech. Project — Final Evaluation

November 2026

Outline

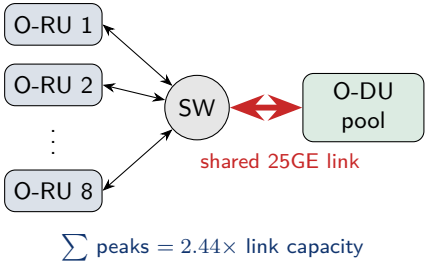
1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

What is the fronthaul and why is it loaded?

- **Fronthaul** = link between the radio unit on the tower (**O-RU**) and the processing unit (**O-DU**).
- In the O-RAN **7-2x split** it carries frequency-domain **IQ samples** for every scheduled resource block (PRB).
- Rates are huge: raw 64-antenna cell $\approx$ **15.4 Gbit/s**; 273 PRB $\times$ 4 layers at BFP-9: **7.4 Gbit/s** peak.
- 7-2x load *follows the scheduled traffic* $\Rightarrow$ operators **over-subscribe** one Ethernet link across many cells.



Problem

When bursts of several cells coincide, the link cannot carry everything; low-latency cells tolerate only $\sim$2 ms of waiting.

Two levers for fronthaul load management

Part A: make the data smaller

- Compress the per-slot IQ matrix using its structure (low rank, delay-domain sparsity)
- Truncated SVD, RAS-BFP (sketch + QR), CSEE
- Rank-adaptive functional split (ACAFS)
- Inversion benchmark + RZF precoder

C/AVX-512/OpenMP kernels for a 0.5 ms slot

Part B: share one link well

- Several cells share a link smaller than their peak
- Every 10 ms decide each cell's bit budget *before* the traffic is known
- Simulator, 7+ reference controllers
- **New rule**: quantile forecast + KKT split

A + B: the compression width sets *how much* load reaches the link; the allocation rule decides *who* loses data when bursts coincide.

Objectives and contributions

1. **Part A — Compression and computation**
   - Two structure-aware IQ encoders (**CSEE**, **RAS-BFP**) with error bounds, compared to O-RAN BFP and truncated SVD on 3GPP TDL-A channels.
   - A rank-adaptive split controller (**ACAFS**).
   - CPU benchmark of 9 matrix-inversion methods incl. learned inverters.
   - Complex RZF precoding: RMT-guided warm-start inversion and a **fused C/AVX-512/OpenMP precoder kernel** (10–15$\times$ over numpy+LAPACK).
2. **Part B — Shared-link budget allocation** (main work)
   - Forecast the **deadline-feasible rate** r^* of each cell, not its volume.
   - Closed-form **KKT (constrained newsvendor) rule** on a quantile forecast, with 4 propositions.
   - Fair evaluation: 12 controllers, 2 traffic families, 20 test seeds, paired statistics.
3. **Extended study** — MDP formulation, Markov-fluid theory of r^*, a 3rd traffic family, 7 more controllers.
4. **Outputs** — NCC 2027 paper (6 pp.), 11-page extended manuscript, fully reproducible code (38 tests).

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

Part A system model

Received frequency-domain matrix after OFDM demodulation:

$$Y = H \odot X + W \in \mathbb{C}^{M \times N}$$

- $M = 64$ antennas, $N = 1200$ subcarriers (100 MHz class)
- 3GPP TR 38.901 **TDL-A** delay profile
- Exponential spatial correlation across antennas

Raw fronthaul rate (32-bit IQ, numerology 0):

$$R_{\text{raw}} = 2MN_{\text{sc}}N_{\text{sym}}\, b\, R_{\text{slot}} \approx 15.36 \text{ Gbit/s/cell}$$

Two structures to exploit

- **Low rank**: correlated antennas $\Rightarrow$ numerical rank $r \ll \min(M, N)$
- **Delay sparsity**: few strong channel taps $\Rightarrow$ sparse after IFFT along subcarriers

Baseline: O-RAN BFP

Shared block exponent + b-bit mantissa per PRB. Only shortens numbers: $\approx 4\times$.

Compression methods

CSEE: delay-domain sparse encoder

$Y_d = \mathrm{IFFT}_N(Y), \;\; S = \mathrm{TopK}(|Y_d|),$
$\hat{Y} = \mathrm{FFT}(Y_d|_S)$

Keep top-K taps, per-antenna BFP.

Thm 1: $\mathrm{NMSE} \le \dfrac{\sigma^2_{\mathrm{tail}}}{\sigma^2_Y} + \dfrac{\Delta^2}{12}\dfrac{K}{MN\sigma^2_Y}$

RAS-BFP: randomised sketch + QR

1. SRHT sketch $Z = \Omega Y \in \mathbb{C}^{r\times N}$
2. Thin QR: $Z^H = QR; \;\; L = YQ$
3. Send $\mathrm{BFP}(L), \mathrm{BFP}(Q); \;\; Y \approx LQ^H$

Thm 2: $\|Y - LQ^H\|_F^2 \le (1+\varepsilon_r)\|Y - Y_r\|_F^2$ w.p. $\ge 1-\delta$

Thm 4: $r^\star = \left\lfloor \sqrt{B_{\mathrm{total}}/(b(M+N))} \right\rfloor$

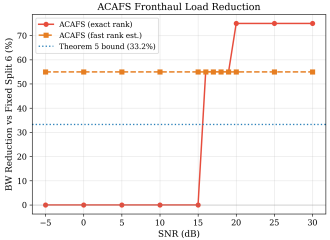
Method	Complexity	Theory
O-RAN BFP	$\mathcal{O}(MN)$	empirical quantisation
Truncated SVD	$\mathcal{O}(MN\min(M,N))$	Eckart–Young
CSEE	$\mathcal{O}(MN\log N)$	Theorem 1
RAS-BFP	$\mathcal{O}(rMN)$	Theorems 2–4 (Halko–Martinsson–Tropp)

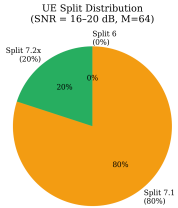
ACAFS: rank-adaptive functional split

Per-UE rank fraction r_k/M chooses the split:

Rank fraction	Split
$r/M < 0.15$	7.2x
$0.15 \le r/M < 0.55$	7.1
$r/M \ge 0.55$	6

Uniform ranks: $\approx 33\,\%$ cut ($\tau_\ell = 0.15$, $\tau_h = 0.55$).
Simulated: **0 %** below $\sim$15 dB SNR, **55–75 %** above (rank collapse at high SNR).

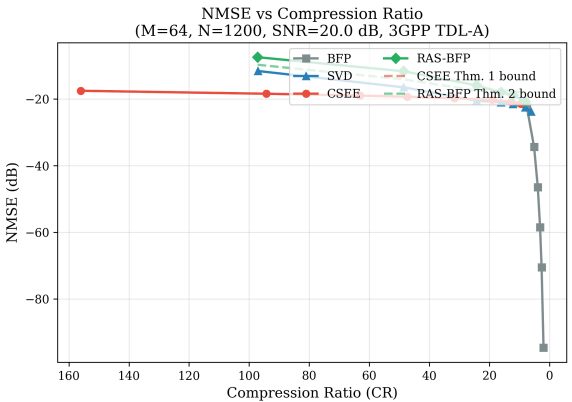




Theorem 5 (bandwidth cut vs. fixed Split 6)

$$\frac{\Delta \mathrm{BW}}{\mathrm{BW}_6} = 1 - \left(\mathrm{BW}_{7.2x}\,\tau_\ell + \mathrm{BW}_{7.1}(\tau_h - \tau_\ell) + \mathrm{BW}_6(1 - \tau_h)\right)$$

Part A results: compression ratio vs. error

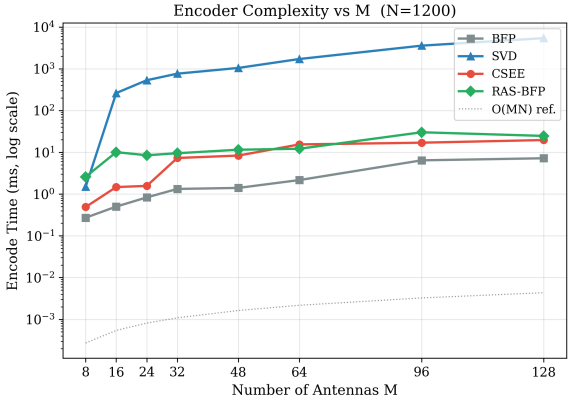


$M = 64$, $N = 1200$, SNR 20 dB, TDL-A

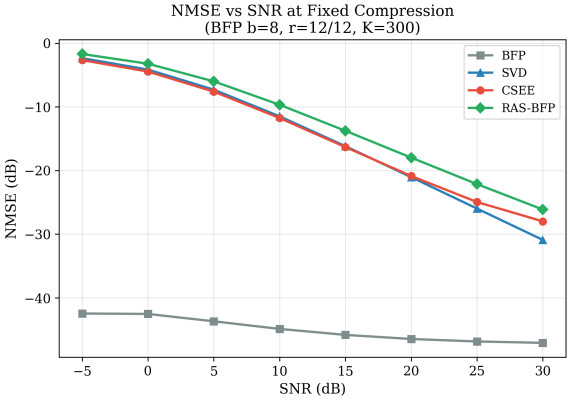
Method	NMSE	CR	Encode
BFP ($b = 8$)	-46 dB	$3.9\times$	3.1 ms
SVD ($r = 12$)	-21 dB	$16.2\times$	18–30 ms
CSEE ($K = 300$)	-21 dB	$12.7\times$	**5.2 ms**
RAS-BFP ($r = 12$)	-18 dB	$16.2\times$	**7.2 ms**

- Structure-aware encoders reach **13–16**$\times$ at ≈ -20 dB vs. $\sim 4\times$ for BFP.
- RAS-BFP **3–4**$\times$ **faster** than SVD; CSEE **4–6**$\times$ **faster**.
- CSEE with $K = 60$: $\sim$63$\times$ at -19 dB (Pareto knee).

Part A results: complexity and SNR robustness



Encode time scaling: SVD grows fastest in M.



NMSE stable across SNR $\in [-5, 30]$ dB.

Matrix-inversion benchmark: why and how

Why? MMSE equalisation, ZF precoding, channel estimation and the encoders all need inverses/decompositions *every slot* (0.5 ms).

Dataset

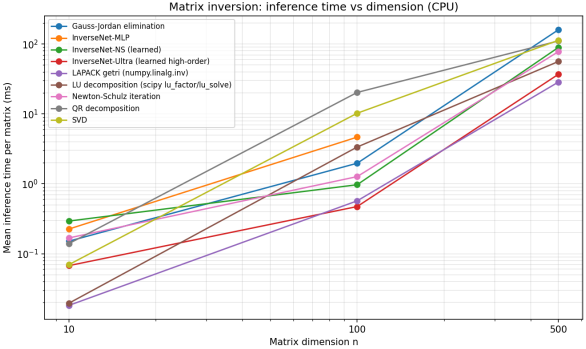
- $A = G/\sqrt{n} + 2I$, $G_{ij} \sim \mathcal{N}(0,1)$ (well conditioned)
- $n \in \{10, 100, 500\}$, 1000 samples each, float32
- 80 % train (neural models), timing on **200 held-out**
- Per matrix (batch 1), warm-up, CPU only

Methods

- **Classical**: LAPACK getri, LU (scipy), QR, SVD, Gauss–Jordan, Newton–Schulz
- **InverseNet-MLP**: flatten $\rightarrow$ regress A^{-1} directly (cannot scale: $> 2 \cdot 10^9$ params at $n = 500$)
- **InverseNet-NS**: unrolled Newton–Schulz, 8 steps, learned step coefficients
- **InverseNet-Ultra**: learned high-order iteration

Metrics: median / p95 time, relative error $\|X - A^{-1}\|_F / \|A^{-1}\|_F$.

Matrix-inversion benchmark: results



Median time [ms] and relative error (held-out)

Method	n=100	n=500	rel. err
LAPACK getri	**0.17**	**18.0**	$5{\cdot}10^{-8}$
InverseNet-Ultra	0.46	33.8	$1{\cdot}10^{-6}$
InverseNet-NS	0.95	92.1	$4{\cdot}10^{-4}$
Newton–Schulz	1.02	70.3	$2{\cdot}10^{-7}$
SVD	2.76	106.9	$1{\cdot}10^{-7}$
LU (scipy)	2.87	56.3	$1{\cdot}10^{-7}$
QR	3.94	102.2	$3{\cdot}10^{-7}$
InverseNet-MLP	4.58	n/a	**0.52**

Part A: lessons

- **LU via LAPACK** is the method to beat: fastest exact method at every size.
- **Slot budget**: $n \approx 100$ fits a 0.5 ms slot on one CPU core; $n \approx 500$ (18 ms) does not $\Rightarrow$ invert per coherence interval, track incrementally, or offload.
- **Direct neural inverse fails** ($\sim$50 % error): the matrix $\rightarrow$ inverse map is non-smooth with n^2 outputs.
- **Learned iteration works** (10^{-6} error): keep the Newton–Schulz structure, learn only coefficients; fixed, predictable cost of batched matmuls (GPU/NPU friendly).
- **Predictability matters**: LAPACK p95 can be 5–$10\times$ the median under thread contention; fixed-step methods are flat.

Theme carried into Part B

Keep the mathematical structure and **learn only what is uncertain**.

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

From real benchmarks to the real workload: RZF precoding

Gaps in the first benchmark

- Real, i.i.d., well-conditioned $A = G/\sqrt{n} + 2I$; precoding needs **complex Hermitian** Grams of **correlated, near-far** channels.
- "InverseNet-Ultra beats LAPACK" holds only for the mean, not the median.
- The slot-to-slot warm start proposed in `REPORT.md` was never measured.
- No link from inversion to the fronthaul.

New study (`cmatinv/`, `experiments_v2/`)

- 38.901 UMa near-far, TDL-A, spatial correlation, Rician LoS, Jakes aging
- M=64, K=16, 273 PRBs, 0.5 ms slots, 3.5 GHz

Per-PRB RZF precoder (every slot)

$$G = HH^H + \alpha I \in \mathbb{C}^{K \times K}$$

$$W = H^H G^{-1} \in \mathbb{C}^{M \times K}$$

$\times 273$ PRBs inside a **0.5 ms** slot.

Question

Can the whole precoder (Gram + inverse + W) fit in a slot on a CPU, and can a warm-started iteration make the inverse cheaper?

RMT-WS: random-matrix-guided warm-start inversion

1. **Equilibrate** $\hat{G} = D^{-1/2} G D^{-1/2}$ (removes the near-far spread).
2. **Eigensolve-free spectral interval** (Marchenko–Pastur + moment match): $v = \|\hat{G}\|_F^2/K - 1$, $[a, b] = 1 + v/\bar{s} \mp 2\sqrt{v}$ (cost K^2).
3. **Cold start**: Chebyshev polynomial on $[a, b]$, then Newton–Schulz $X \leftarrow X(2I - \hat{G}X)$.
4. **Tracking**: next slot starts from the previous inverse.
5. **Residual certificate** $\|(I - \hat{G}X)z\|/\|z\|$ (2 probes, $4K^2$ MACs) picks the number of Newton steps, or falls back to a cold start when $r > 0.3$.

Doppler-to-warm-start law

$$r_0 \approx c \cdot 2\sqrt{\tfrac{K}{M-K}}\sqrt{1-\rho}, \quad 1 - \rho \approx (\pi f_D T)^2$$

r_0 grows **linearly with speed** (measured slope 1.00). Warm start valid up to $\approx$ **39 km/h** (M=64, K=16), 60 km/h (K=8), 23 km/h (K=32).

Certificate safety

Tail bound $\Pr(\hat{r}^2 \le (1-d)r^2) \le e^{-pd^2 s/2}$; audit of 480 cases: **0** accepted-but-non-contractive at $\hat{r} \le 0.3$.

Inversion results: operation count and tracking

GEMMs for $\leq 1\,\%$ sum-rate loss (cold start)

Method	64/16	128/16	64/32
Neumann (Wu/Prabhu)	> 8	> 8	> 8
Joint Newton-Neumann	> 10	> 10	> 10
Newton–Schulz, classic	> 10	> 10	> 10
Cheb-MP + NS (ours)	**6**	5	**9**
Gauss–Seidel	**6**	**4**	> 12
Block CG	7	5	12

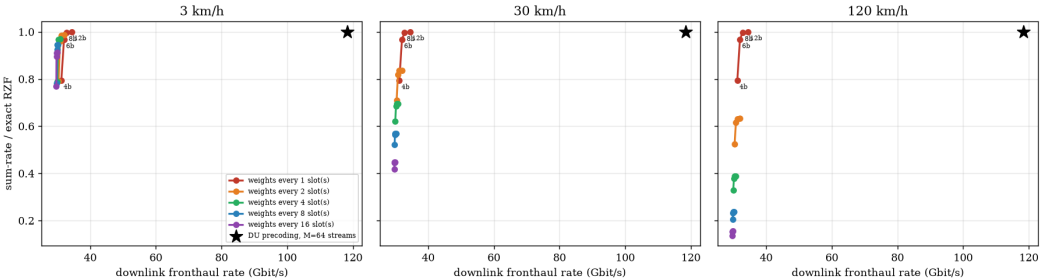
Beats Neumann-type schemes; **ties or loses** to Gauss–Seidel/CG at $K/M \leq 0.25$. An exact Cholesky/LU inverse costs ≈ 1 GEMM.

Tracking, $K{=}16$, $M{=}64$, 95 % CI over 16 drops

Speed	Cold start (6 GEMM)	**RMT-WS**
3 km/h	0.41 %	**0.011 % @ 2.25**
30 km/h	0.41 %	0.62 % @ 5.4
120 km/h	0.39 %	0.39 % @ 6.25

- 3 km/h: **2.7× fewer GEMMs**, 30× lower loss.
- 30 km/h: a wash; 120 km/h: becomes a cold start (never diverges, unlike fixed warm starts).
- Low precision (8-bit mantissa, wide accumulate): 0.15 % vs 0.88 % for direct Gauss–Jordan.

Where should precoding run? Fronthaul impact



Architecture (100 MHz, M=64, K=16, BFP-9 IQ)	Fronthaul	Sum-rate vs exact RZF
DU precodes, sends $M = 64$ streams	118.4 Gbit/s	100 %
RU precodes, $K = 16$ layers + 8-bit weights every slot	**32.8 Gbit/s (-72 %)**	99.7 %
RU precodes, $K = 16$ layers + 6-bit weights every slot	32.0 Gbit/s (-73 %)	96.7 %

At mobility the weights must be refreshed **every slot** (every 4 slots: 70 % rate at 30 km/h) — exactly the workload a fast per-slot precoder kernel serves. Caveat: saving assumes 3-PRB weight groups; per-subcarrier weights would exceed the DU-side rate.

Kernel programming: a fused RZF precoder in C

`kernels/cmat_kernels.c` (batched complex64, Python binding `cmatinv/fastkernels.py`)

- **One pass per PRB**: load H once $\rightarrow$ Gram (lower triangle) $\rightarrow$ inverse $\rightarrow W^T$ + power scaling; all stack/L1-resident, no DRAM intermediates.
- **Split re/im planes** (SoA) so complex arithmetic vectorises.
- **OpenMP** `parallel for` across the 273 PRBs; `omp simd` inner loops.
- **Gauss–Jordan sweep inverse**: K rank-1 updates, contiguous fully-vectorised rows, no sqrt/divide chains; no pivoting needed since $G \succ 0$ (Cholesky kept behind `-DUSE_CHOL`).
- **Register-blocked** $W = H^H X$; **AVX-512** codegen; `rdtsc` profiling.
- **Fused Newton step** returns the *exact* residual $\|I - GX\|_F / \sqrt{K}$ for free.

Inner loop of the sweep inverse

```
for (int i = 0; i < K; i++) {
  if (i == k) continue;
  float fr = ckr[i]*piv, fi = cki[i]*piv;
  float *ar = xr + i*K, *ai = xi + i*K;
  #pragma omp simd
  for (int j = 0; j < K; j++) {
    ar[j] -= fr*rkr[j] - fi*rki[j];
    ai[j] -= fr*rki[j] + fi*rkr[j];
  }
  ar[k] = -fr; ai[k] = -fi;
}
```

Build (`kernels/build.sh`)

```
gcc -O3 -march=native -fopenmp -ffast-math
    -mprefer-vector-width=512 -shared -fPIC
```

Kernel optimisation log (profiling-driven)

Per-stage cycles per PRB (K=16, M=64, 1 thread):

1. **Fusion + Hermitian symmetry**: Gram on lower triangle only.
2. **Register-blocked axpy** $W = H^H X$, output $W^T[S, K, M]$: 0.81 → **0.42 ms**/slot.
3. **GJ sweep instead of Cholesky**: inverse 6.4k → **2.9k** cycles (K=16); 21k → 11k (K=32).
4. **AVX-512** (`-mprefer-vector-width=512`; gcc uses 256-bit by default): 1.3–1.5× single-thread, 2× on 4 threads.
5. 4-column blocked Gram: **slower, reverted**.

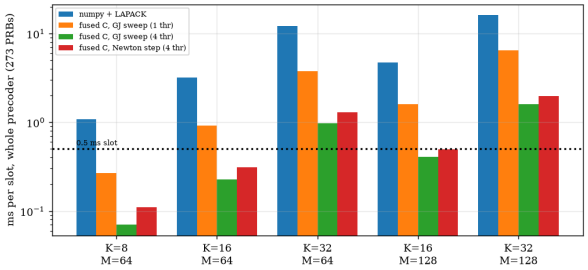
Where the cycles go after tuning

- `form_w` ($K \times K$ by $K \times M$): **44–58 %**, ≈30 flops/cycle (∼half AVX-512 peak)
- Gram: 20–24 %, ≈28 flops/cycle
- Inverse: the rest

Accuracy

float32 kernel vs float64 exact: element error $\approx 3 \cdot 10^{-7}$, sum-rate loss $< 10^{-5}$ % (incl. ill-conditioned, K=32, 30 dB).

Kernel results: whole precoder per slot (273 PRBs)

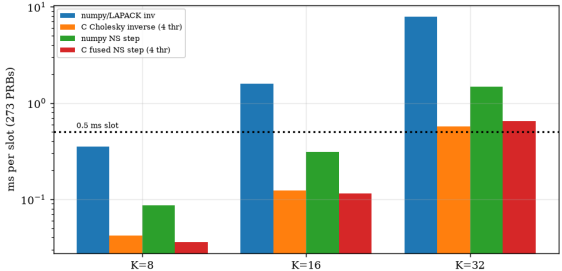


4-core AVX-512 Xeon, complex64, median of 40 runs

K,M	numpy+LAPACK	C 1 thr	C 4 thr	gain
8, 64	1.08 ms	0.27	**0.070**	15×
16, 64	3.21 ms	0.92	**0.226**	14×
32, 64	12.2 ms	3.75	0.97	12.5×
16, 128	4.71 ms	1.61	**0.41**	11.6×
32, 128	16.3 ms	6.43	1.60	10×

- $K = 8, 16$ (M=64) and K=16, M=128 now **fit a 0.5 ms slot** for the *entire* precoder (none did before).
- Near-linear thread scaling: 0.92 → 0.45 → 0.23 ms (1/2/4 threads).
- The Newton-step path (0.31 ms) is slower than the exact GJ sweep (0.23 ms) inside the fused pipeline.

Kernel programming: honest corrections and negative results



Inverse only: tuned C Cholesky vs fused NS step (4 threads)

Verdict

Solid kernel engineering (10–15×, slot-feasible precoder), not research novelty. Recommended CPU path: the **fused exact kernel**.

- **Speed-up is mostly threading**: single-thread gain is 2.5–4×; batched `numpy.linalg.inv` is single-threaded. MKL batched / Eigen not compared.
- **A good exact inverse wins on CPU**: NS step 0.115 vs Cholesky 0.124 ms ($K{=}16$), 0.66 vs 0.58 ms ($K{=}32$), and tracking needs 2.25–6.25 steps. Newton/certificate only pays on GEMM-engine hardware (GPU/NPU/systolic), not measured.
- **AVX512-FP16 weight formation** (`_mm512_fcmadd_pch`): **not faster** (0.78–1.30×) and costs 0.50 % sum-rate at 30 dB ⇒ rejected, kept as documented negative result.
- AMX, cache-blocking for $K > 32$, NUMA pinning, real-time OS settings: not done.

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

Problem setting

- $N = 8$ cells: **3 low-latency (LL)** ($D = 2$ ms, priority 3) and **5 eMBB** ($D = 10$ ms, priority 1).
- One 25GE aggregation link; 3 % reserved for C/M/S-plane
  $\Rightarrow C = 12.125$ Mbit per 0.5 ms slot.
- Aggregate cell peak $= 2.44\,C$ (statistical multiplexing).
- Every $T = 20$ slots (10 ms) a coordinator sets per-cell budgets r_i that hold for the whole interval.
- Telemetry is $\tau = 2$ slots (1 ms) old.

Why decide in advance?

- Near-RT RIC / E2 loops run at $\geq$10 ms
- Switch shaping is not reconfigured per slot
- DU schedulers cannot consult each other within the sub-ms pipeline

$\Rightarrow$ reservation lead time $H = T + \tau$ slots.

Question

If each cell's need for the next interval is available as a forecast *distribution*, how should the shared link be split between cells with different deadlines and priorities?

System model

Fronthaul cost of one PRB-layer per slot, b-bit BFP (this is where Part A enters):

$$\beta(b) = (12 \cdot 2 \cdot b + 8) \cdot 14 \cdot \kappa$$

$\beta(9) = 3387$ bits ($\kappa = 1.08$); BFP-6: $0.68\times$, BFP-12: $1.32\times$, BFP-14: $1.54\times$.

Fronthaul demand of cell i (user bits a_i, spectral efficiency η_i):

$$d_i(t) = \lceil a_i(t)/\eta_i(t) \rceil \, \beta$$

Load follows scheduled PRB-layers, not application bits.

Decision every T slots:

$$\sum_i r_i \le C, \qquad 0 \le r_i \le \mathrm{cap}_i = 273 L_i \beta$$

- FIFO DU queue per cell; bits waiting $> D_i$ slots are dropped = **deadline violations**.
- Link never queues (budgets enforce capacity).

Primary metric (fixed before tuning): priority-weighted violation ratio

$$V_w = \frac{\sum_i w_i^{\mathrm{p}} \, \mathrm{violated}_i}{\sum_i w_i^{\mathrm{p}} \, \mathrm{arrived}_i}$$

Key idea 1: forecast the deadline-feasible rate r^*

For arrivals $a_1, \ldots, a_H$ and deadline D, the smallest constant rate at which **no bit misses its deadline**:

$$r^*(a, D) = \max_{1 \le s \le e \le H} \frac{\sum_{t=s}^{e} a_t}{e - s + 1 + D}$$

- Network-calculus service-curve condition; tight (verified to 1 PRB-layer in tests).
- $D \gg H$: $r^* \to$ mean rate. $D \ll H$: $r^* \to$ peak burst rate.
- For a 2 ms class: r^* = peak 2 ms burst rate, often 2–$3\times$ the mean and **strongly right-skewed**.

Demand of every deadline-aware controller

$$\text{need}_i = B_i + r_i^*, \qquad B_i = \frac{\text{backlog}_i}{\text{slack}_i}$$

Controllers differ only in *how* r_i^* is estimated and how a shortage is split.

A single predicted number is a poor basis for reserving a skewed quantity $\Rightarrow$ forecast a **range**.

Quantile forecaster

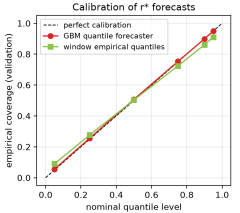
- Target: $y_i = r^*(d_i[t-\tau : t+T], D_i)/\mathrm{cap}_i$
- **28 features**: last 6 horizon means (+ mean/std/max), 2 EWMAs, last 4 slot demands, r^* of last 3 horizons, relative spectral efficiency, D_i/H, cell one-hot. Normalised by config constants (no fitted statistics).
- **Model**: one HistGradientBoosting quantile regressor per level $\alpha \in \{0.05, 0.25, 0.5, 0.75, 0.9, 0.95\}$, 300 trees, 15 leaves.
- Train: 16 traces $\times$ 16k slots; 101k rows; **12 s** on CPU.
- Seeds: train 1000–1015, val 2000–2003, test 3000–3019 (disjoint).

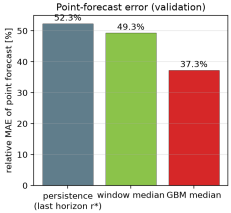
Validation quality

Coverage at nominal 5/25/50/75/90/95 %:
5.6 / 25.6 / 50.7 / 75.3 / 89.9 / 94.9 %

Median rel. MAE: **37 %** (GBM)
vs. 49 % window median, 52 % persistence

Pinball loss: 0.0224 vs. 0.0293





Key idea 2: the KKT (constrained newsvendor) allocation rule

Surrogate objective (one interval, forecast distribution F_i of r_i^*):

$$\max_{r} \sum_i w_i \, \mathbb{E}\big[\min(B_i + r_i^*,\ r_i)\big] \quad \text{s.t. } \sum_i r_i \le C,\ 0 \le r_i \le \mathrm{cap}_i$$

$\mathbb{E}[\min(\cdot, r)]$ is concave with derivative $w_i\,(1 - F_i(r - B_i)) =$ weighted probability that the marginal bit is needed. Separable concave $\Rightarrow$ KKT gives the global optimum:

$$\boxed{r_i(\lambda) = \mathrm{clip}\Big(B_i + F_i^{-1}\big(1 - \lambda/w_i\big),\ 0,\ \mathrm{cap}_i\Big)}$$

- One scalar $\lambda \ge 0$ with $\sum_i r_i(\lambda) = C$: **60-step bisection**; $\lambda = 0$ if the link is not binding (surplus shared as headroom).
- F_i^{-1}: piecewise-linear through the 6 quantiles; cost $\mathcal{O}(NK + Nn_{\mathrm{iter}})$, tens of μs for $N = 8$.

Intuition

Every cell is cut at the **same priority-weighted risk**. Wide forecast or high priority $\Rightarrow$ more margin; confident forecast $\Rightarrow$ about its median. Margins are *derived*, not tuned.

Properties of the rule

Proposition 1 (feasibility)

For any forecast and any λ, budgets satisfy the caps and $\sum_i r_i \le C$.

Proposition 3 (backlog first)

Any cell with $\lambda < w_i$ receives at least $\min(B_i, \mathrm{cap}_i)$ before uncertain demand of an equal- or lower-weight cell is funded.

Proposition 2 (equal weighted tail probability)

At an interior optimum $w_i\big(1 - F_i(r_i - B_i)\big) = \lambda$ for every cell. Point forecasts (degenerate F_i) reduce the rule to a priority-ordered fill.

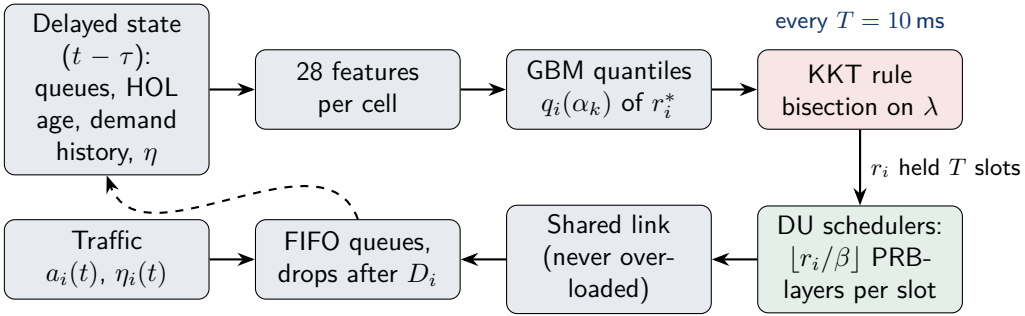
Proposition 4 (robustness to miscalibration)

If $\sup_x |F_i(x) - G_i(x)| \le \epsilon$ (Kolmogorov) for every cell, then

$$J_G(r^G) - J_G(r^F) \le \epsilon \sum_i w_i \big[(r_i^G - B_i)^+ + (r_i^F - B_i)^+\big] \le 2\,\epsilon\, w_{\max}\, C.$$

Loss from a miscalibrated forecast grows at most **linearly** in ϵ.

Control loop



- **Edge cases**: non-finite forecasts → window quantiles; zero demand → zero budget + headroom; link not binding → full predictive support.
- **Calibrated variant**: adaptive conformal offset $\delta_i \leftarrow \mathrm{clip}(\delta_i + \gamma(\mathbb{1}[y_i > q_i(0.9)] - 0.1))$, reported as ablation.

Reference controllers (all tuned on validation seeds)

Controller	Estimate of r^*	Allocation
Static equal	none	C/N
Reactive proportional	mean-rate EWMA (deadline-unaware)	proportional water-fill
MaxWeight	priority $\times$ queue, HOL-age urgency	max-weight
Deadline-aware reactive	persistence: last observed r^*	deadline-weighted water-fill
Point forecast	GBM **median**	deadline-weighted water-fill
Fixed global quantile	GBM quantile, one tuned level (0.75)	water-fill
Proposed (window)	empirical quantiles, last 30 horizons (**no ML**)	KKT rule
Proposed	**GBM quantiles**	**KKT rule**
Proposed + ACI	GBM quantiles + conformal offset	KKT rule
Oracle water-fill	true r^* (not implementable)	water-fill
Oracle + KKT	true r^* (not implementable)	KKT rule

- Margins $\{0, 0.25, 0.5\}$, EWMA windows, persistence windows tuned; **margin 0 was best for all** (inflating a point estimate hurts under proportional shortage split).
- Every baseline redistributes unused capacity; same weights, backlog rate and water-fill as the point-forecast controller.

Evaluation protocol

Family S (synthetic, bursty)

- Gamma background + ON/OFF bursts with **Pareto sojourns** ($\alpha_{\text{on}} = 1.5$, $\alpha_{\text{off}} = 1.8$), slow regimes, SE drift
- Scenarios: low 0.50, moderate 0.65, high 0.80, flash crowds, **held-out shift** (never seen in training/tuning)

Family G (standards-based, nothing hand-designed)

- 3GPP TR 38.901 UMa, 7 interfering sites, Jakes fading, TS 38.214 CQI table
- eMBB: FTP Model 3; LL: XR video (TR 38.838, 60 fps)
- Demand = output of a round-robin PRB scheduler

Protocol

- 20 test seeds $\times$ 20,000 slots (10 s) after 1,000 warm-up
- **Identical traces** and identical stale observations for all controllers
- Separate train / validation / test seeds; forecaster frozen before test
- Paired per-seed comparison: bootstrap 95 % CI, **Holm-corrected Wilcoxon** tests
- Ablations: point vs. quantile, ML vs. non-ML, calibration on/off, T, τ, miscalibration, priority weight

Supporting metrics: per-class violations, utilisation, unused allocation, DU delay, Jain fairness, decision time.

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

Main result, Family S: V_w [%] (mean $\pm$ std, 20 seeds)

Method	low 0.50	moderate 0.65	high 0.80	flash crowds	shift (held-out)
Static equal share	4.45 $\pm$ 1.05	9.65 $\pm$ 1.36	15.46 $\pm$ 1.51	19.15 $\pm$ 2.34	22.31 $\pm$ 2.99
Reactive proportional	0.83 $\pm$ 0.27	4.11 $\pm$ 0.69	9.20 $\pm$ 0.99	12.27 $\pm$ 2.23	11.70 $\pm$ 2.24
MaxWeight	0.90 $\pm$ 0.28	4.85 $\pm$ 0.79	11.54 $\pm$ 1.27	15.24 $\pm$ 3.02	12.31 $\pm$ 2.44
Deadline-aware reactive	0.52 $\pm$ 0.21	4.24 $\pm$ 0.78	11.03 $\pm$ 1.33	14.51 $\pm$ 3.06	12.56 $\pm$ 2.81
Point forecast (GBM median)	0.27 $\pm$ 0.17	3.23 $\pm$ 0.73	9.70 $\pm$ 1.34	13.33 $\pm$ 3.09	11.30 $\pm$ 2.81
Fixed global quantile (0.75)	0.34 $\pm$ 0.23	4.35 $\pm$ 1.02	11.76 $\pm$ 1.44	15.09 $\pm$ 3.17	12.85 $\pm$ 3.32
Proposed, window quantiles (no ML)	0.25 $\pm$ 0.13	2.57 $\pm$ 0.64	8.72 $\pm$ 1.34	12.67 $\pm$ 3.19	10.22 $\pm$ 2.88
Proposed (GBM quantiles + KKT)	**0.15 $\pm$ 0.13**	**2.15 $\pm$ 0.60**	**8.00 $\pm$ 1.27**	**11.76 $\pm$ 3.08**	**10.04 $\pm$ 2.71**
Proposed + ACI calibration	0.17 $\pm$ 0.13	2.17 $\pm$ 0.61	8.14 $\pm$ 1.30	12.27 $\pm$ 3.26	11.12 $\pm$ 3.22
Oracle water-fill (not implementable)	0.07	1.62	6.89	10.16	6.69

- The proposed rule is the **best implementable controller in all 5 scenarios**.
- Std across seeds is dominated by burst timing that all methods share $\Rightarrow$ the **paired** comparison is the right reading (next slide).

Paired per-seed reduction by the proposed rule (Family S)

Mean relative change of V_w [95 % bootstrap CI], seeds won /20; ** Holm-Wilcoxon $p < 0.01$, * $p < 0.05$

Proposed vs.	low	moderate	high	flash crowds	shift
Reactive prop.	−81 % 20/20**	−48 % 20/20**	−13 % 20/20**	**−4 % 16/20***	−14 % 20/20**
MaxWeight	−83 % 20/20**	−56 % 20/20**	−31 % 20/20**	−23 % 20/20**	−18 % 20/20**
Deadline-aware reactive	−70 % 20/20**	−49 % 20/20**	−27 % 20/20**	−19 % 20/20**	−20 % 20/20**
Point forecast	−43 % 20/20**	−33 % 20/20**	−18 % 20/20**	−12 % 20/20**	−11 % 20/20**
Fixed global quantile	−54 % 20/20**	−50 % 20/20**	−32 % 20/20**	−22 % 20/20**	−22 % 20/20**
Window quantiles (no ML)	−37 % 20/20**	−16 % 20/20**	−8 % 20/20**	−7 % 20/20**	**−2 % 16/20****
+ ACI calibration	−9 % 18/20**	−1 % 16/20**	−2 % 19/20**	−4 % 20/20**	−10 % 20/20**

Supported

vs. point forecast, fixed quantile, MaxWeight and deadline-aware reactive: every scenario incl. held-out shift, **every seed**, $p < 0.01$.

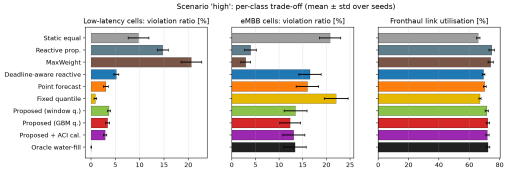
Weak

vs. reactive proportional under flash crowds (-4 %): it starves LL cells (17.8 % vs 4.6 %) but serves eMBB better.

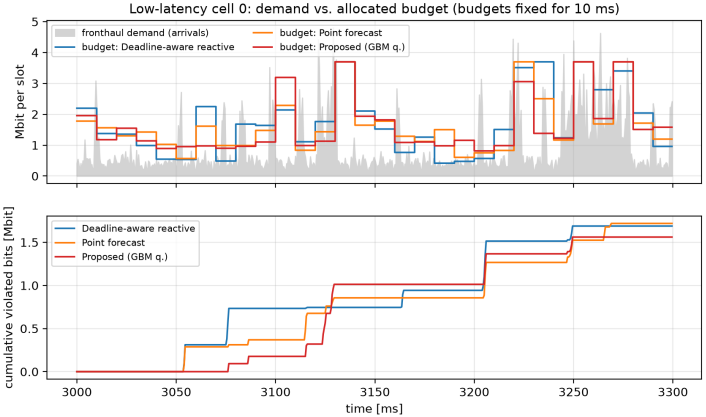
Where the gain comes from (scenario high, 20 seeds)

Method	LL viol.	eMBB viol.	util.	Jain
Reactive prop.	14.7 %	4.0 %	74.7 %	0.996
MaxWeight	20.6 %	2.9 %	74.1 %	0.991
Deadline-aware react.	5.2 %	16.5 %	69.3 %	0.995
Point forecast	3.1 %	16.0 %	70.1 %	0.994
Fixed global quantile	0.9 %	22.0 %	67.0 %	0.985
Proposed	3.4 %	**12.3 %**	**72.2 %**	0.997
Oracle water-fill	0.04 %	13.4 %	72.3 %	0.994

- Deadline-unaware controllers sacrifice LL cells: large queues $\neq$ urgent queues.
- In the deadline-aware family the proposed rule has the **lowest eMBB loss and highest utilisation**: margin only where the forecast is wide.
- One global level over-protects LL and wastes 33 % of the allocation.



The rule in action: one low-latency cell



- Grey: demand; steps: budgets held for 10 ms.
- The reactive controller is **one interval late** at every burst.
- The proposed rule raises the budget where the forecast range is **wide** (burst onset).
- Fewest cumulative violated bits over the trace.

Ablations: what actually carries the gain?

1. **Point vs. quantile forecast** (same model, same rule family):
   $-11\,\%$ to $-43\,\%$, 20/20 seeds everywhere $\Rightarrow$ **representing uncertainty matters**.
2. **Per-cell KKT level vs. one tuned global level** (same forecast):
   $-22\,\%$ to $-54\,\%$ $\Rightarrow$ the level must be chosen **per cell by the link constraint**.
3. **ML vs. non-ML quantiles** under the same KKT rule:
   GBM adds $-7\,\%$ to $-37\,\%$ on nominal traffic but only $-2\,\%$ under the held-out shift.
   $\Rightarrow$ most of the gain is the **rule** (r^* target + KKT split), not the model. ML is *useful*, not *necessary*.
4. **Online ACI calibration**: **no gain** (1–9 % worse nominal, 10 % worse under shift).
   In a coupled allocation LL misses are partly *caused* by the shortage split $\Rightarrow$ offsets inflate LL reservations and starve eMBB. Per-stream conformal correction does not transfer.

Family G: standards-based 3GPP traffic

Method	mod. 0.65	high 0.80	unweighted (high)
Static equal	10.75	13.10	19.2
Reactive prop.	12.05	15.94	15.0
MaxWeight	17.51	25.79	19.7
Deadline-aware react.	8.95	11.53	18.5
Point forecast	6.00	8.98	16.6
Fixed quantile	6.80	9.88	18.7
Window quantiles	6.18	9.01	16.0
Proposed	**5.08**	**8.01**	**14.4**
Transfer (trained on S)	6.99	9.89	14.3
Oracle + KKT	3.47	6.06	11.1

- Main result **reproduces on traffic we did not design**: best implementable at both loads, 20/20 seeds.
- At high load: $-11\,\%$ vs. point forecast, $-31\,\%$ vs. deadline-aware reactive, $-50\,\%$ vs. reactive proportional.
- Here also best under the **unweighted** ratio.
- Forecaster transferred from Family S still beats every reactive controller, but not the in-family point forecast.

How much is still forecast error? (oracle + KKT)

Scenario	Proposed	Oracle+KKT	Gap
S low	0.15	0.05	70 %
S moderate	2.15	0.88	59 %
S high	8.00	4.77	40 %
S flash crowds	11.76	8.10	31 %
S shift	10.04	5.84	42 %
G moderate	5.08	3.47	32 %
G high	8.01	6.06	24 %

- Oracle + KKT = same rule with a perfect forecast of r^*.
- The gap is what a better forecaster could still buy: **24–40 % at high load**.
- The rest is the unavoidable cost of committing budgets for 10 ms.
- At $T = 2$ ms: oracle + KKT 1.91 % vs. oracle water-fill 6.89 % with the *same* perfect information $\Rightarrow$ **the allocation rule matters as much as the forecast**.
- "Oracle" is a reference, **not** an upper bound.

Honest caveat: the gain is a priority trade

Unweighted violation ratio, Family S [%]

Method	moderate	high	flash
Reactive prop.	**2.44**	**6.55**	**9.64**
MaxWeight	2.73	7.18	10.44
Point forecast	3.93	12.88	17.62
Proposed ($w_{\mathrm{LL}} = 3$)	2.50	10.19	15.08

LL weight sweep, V_w [%], S high

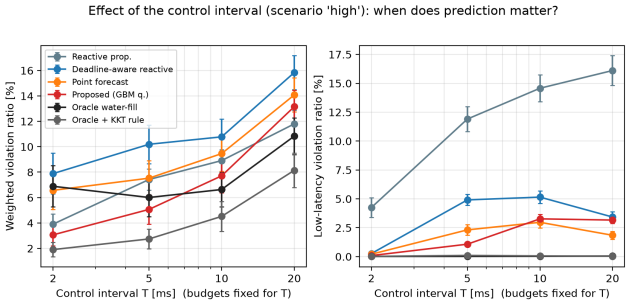
w_{LL}	1	2	3	5
Reactive prop.	**6.55**	**8.13**	9.20	10.56
Point forecast	6.77	8.90	9.70	10.57
Proposed	6.58	8.35	**8.00**	**6.93**

- Under the unweighted ratio, plain proportional sharing loses fewer total bits at moderate/high load (it sacrifices the small LL cells).
- The rule matches proportional at $w_{\mathrm{LL}} = 1$ and pulls ahead from $w_{\mathrm{LL}} = 3$.
- Family G, $w_{\mathrm{LL}} = 1$: rule is *worse* than point forecast (15.9 vs 10.4 %) — zero-inflated XR/FTP demand gives an all-or-nothing newsvendor allocation.

Scope of the claim

For operators who value a low-latency bit at least **2–3**× an eMBB bit.

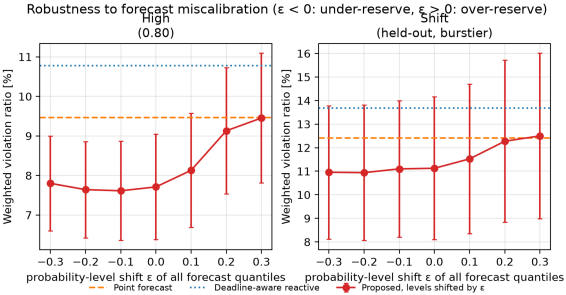
Sensitivity: control interval T and telemetry delay τ



T	React. prop.	Point f.	Proposed
2 ms	3.91	6.57	**3.07**
5 ms	7.42	7.53	**5.07**
10 ms	8.91	9.46	**7.71**
20 ms	**11.80**	14.08	13.15
τ			vs. react. prop.
0 ms	8.16	8.45	6.85 ($-16\,\%$)
1 ms	8.91	9.46	7.71 ($-13\,\%$)
2.5 ms	9.86	10.51	8.77 ($-11\,\%$)
5 ms	11.30	12.45	11.07 ($-2\,\%$)

- Short T: rule beats even the oracle water-fill.
- $T = 20$ ms ($5\times$ LL deadline) or $\tau = 5$ ms: advantage over proportional vanishes.

Robustness to forecast miscalibration (Prop. 4 in practice)



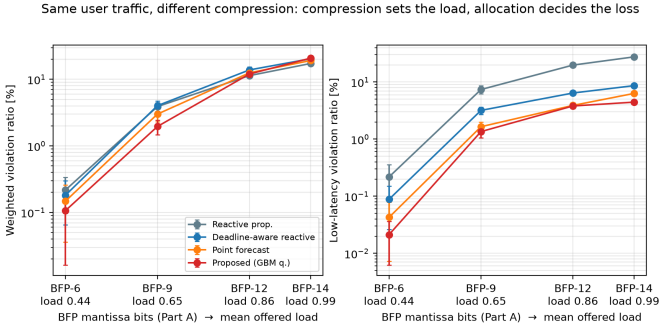
Every forecast quantile level shifted by ϵ before allocation:

ϵ	-0.3	-0.1	0	$+0.1$	$+0.3$
high	7.80	7.61	7.71	8.13	9.45
shift	10.96	11.10	11.13	11.53	12.50

Point forecast: 9.46 / 12.42; deadline-aware: 10.78 / 13.68

- Under-reserving is almost harmless: a common shift is absorbed by λ.
- Over-reserving degrades gradually ($+23\,\%$ at $\epsilon = 0.3$).
- Even then the rule matches the point forecast and beats deadline-aware reactive.

Part A × Part B: compression width vs. allocation rule

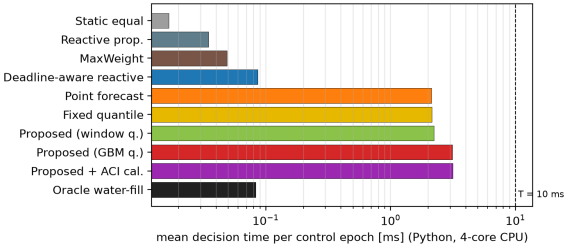


Same user traffic; only BFP width changes β

BFP	load	React.	Point	Prop.
6 bit	0.44	0.22	0.15	**0.11**
9 bit	0.65	3.90	3.00	**1.96**
12 bit	0.86	**11.37**	12.50	12.01
14 bit	0.99	**17.29**	19.16	20.81

- **Compression sets the regime**; allocation decides the loss inside it.
- Allocation matters most at load $\approx$ **0.5–0.8** (half the loss at 0.65).
- At ≈ 1.0 load no split helps: need more compression or capacity.

Decision overhead



Python, 4-core CPU, per 10 ms control epoch:

- Static 0.02 ms, reactive prop. 0.04 ms, MaxWeight 0.05 ms, deadline-aware 0.09 ms
- Window quantiles 2.2 ms, point forecast 2.5 ms, **proposed 3.5 ms**
- The KKT rule itself: < 0.1 ms.
- Cost $=$ features $+ 6 \times 300$ trees; vectorised numpy tree traversal (1.8 ms vs. 12 ms for scikit-learn `predict`).
- Real O-DU: compiled (C++/numba) model or the no-ML window-quantile fallback.

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

The real control problem: a POMDP

State $s(t) = (Q(t), Z(t), \eta(t))$: queue age profiles, hidden traffic state, spectral efficiency. At epochs $t_k = kT$:

$$\min_{\pi} \; \mathbb{E}_{\pi}\Big[\sum_{k} \sum_{t=t_k}^{t_k+T-1} \sum_{i} w_i^{\mathrm{p}} \,\mathrm{dropped}_i(t)\Big], \qquad r(t_k) = \pi\big(s(t_k - \tau), \text{history}\big)$$

Intractable: continuous state of dimension in the hundreds, unknown traffic law, action held for T slots.

Three approximation families, all implemented:

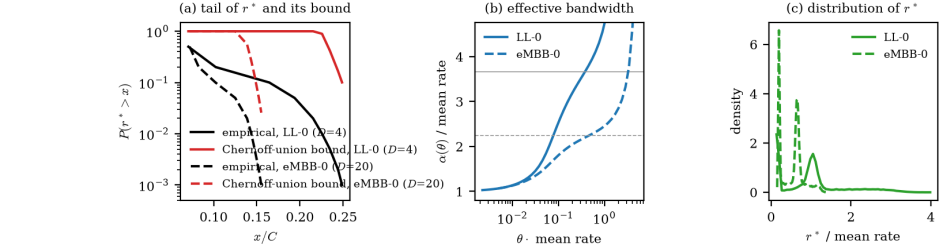
1. **One-step stochastic programs** (certainty-equivalent / rollout): the KKT newsvendor rule on a learned or model-based distribution.
2. **Lyapunov drift**: MaxWeight; **predictive MaxWeight** (Huang et al. 2016) with queue + look-ahead $\times$ forecast median.
3. **Direct policy search**: 3-parameter class-level policy learned by the **cross-entropy method** (CEM) — small analogue of a DRL xApp.

Plus a third traffic family **M**: Markov-modulated Gamma sources (idle/normal/burst), with a **semi-Markov held-out shift** (Pareto sojourns).

Markov-fluid theory of the deadline-feasible rate

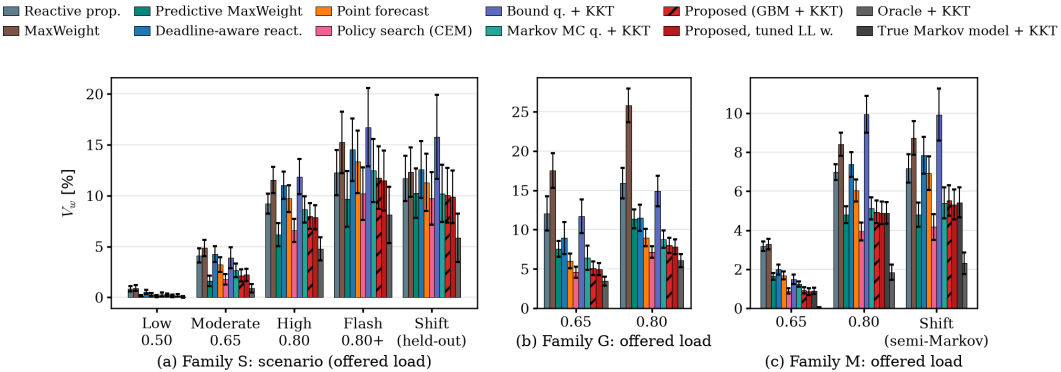
Phase chain P (stationary π), per-phase log-MGF $\psi_k(\theta)$, $\Phi(\theta) = \mathrm{diag}(e^{\psi_k(\theta)})$.

- Window log-MGF: $\Lambda_l(\theta) = \log\left(\pi^{\mathsf{T}}\Phi(P\Phi)^{l-1}\mathbf{1}\right)$; effective bandwidth $\alpha(\theta) = \log\rho(P\Phi(\theta))/\theta$.
- **Chernoff-union bound**: $\Pr(r^* > x) \le \sum_{l=1}^{H}(H-l+1)\min_\theta \exp\big(-\theta x(l+D) + \Lambda_l(\theta)\big)$.
- **Large-deadline decay**: $\limsup_{D\to\infty} D^{-1}\log\Pr(r^* > x) \le -x\,\theta^*(x)$ — links r^* to classical buffer-overflow asymptotics.
- **Shortfall sandwich** (pathwise, any arrivals): $(D+1)(r^*-r)^+ \le V(r) \le \sum_t (a_t - r)^+$.
  $\Rightarrow$ the newsvendor surrogate is a *lower* bound on violated volume.



Validation (Family M, 20k horizons): sandwich never violated; bound valid but **25–100× conservative** at 2–10 ms deadlines $\Rightarrow$ argument for *learning* the distribution.

Three families, nineteen controllers



Extended results: best implementable controller (V_w [%])

Scenario	GBM + KKT	Pred. MaxWeight	CEM policy	Markov-chain KKT	Tuned-w KKT
S low 0.50	0.15	0.14	**0.13**	0.25	0.18
S moderate 0.65	2.15	**1.62**	1.77	2.65	2.21
S high 0.80	8.00	**6.15**	6.57	8.64	7.88
S flash crowd	11.76	**9.69**	10.20	12.44	11.49
S shift	10.04	10.25	**9.73**	10.22	9.90
G moderate 0.65	5.08	7.57	**4.58**	6.44	4.98
G high 0.80	8.01	11.36	**7.17**	8.77	7.83
M moderate 0.65	0.92	1.64	0.88	1.24	**0.85**
M high 0.80	4.94	4.79	**3.93**	5.12	4.88
M shift 0.80	5.52	4.80	**4.17**	5.39	5.31

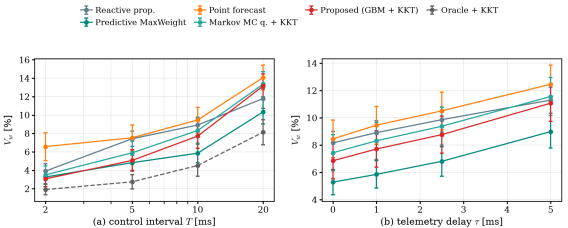
- Within the NCC comparison set the KKT rule still wins **all 10 scenarios** and is the best controller with **nothing to tune**.
- Two controllers using the *same* forecaster beat it: **CEM policy** best in 6/10, **predictive MaxWeight** in 3/10 (all bursty Family S), tuned-weight KKT in 1/10.
- Markov chain fitted online (no training data) is within 3–9 % of the learned rule.

Why does re-tuning the LL weight not close the gap?

KKT multiplier diagnostic (high load, 3 seeds × 8000 slots)

	S	G	M
Link binding	99.8 %	100 %	99.9 %
$\lambda^* \geq w_{\mathrm{eMBB}}$	58 %	56 %	50 %

- In half the epochs every eMBB cell already sits at its lowest quantile: the weight has nothing left to move.
- Tuned weight: 8.00→7.88 (S), 8.01→7.83 (G) — an order less than CEM.
- CEM gain comes from **decoupled per-class levels** ($p_{\mathrm{LL}} \approx 0.8$, $p_{\mathrm{eMBB}} \approx 0.5$) and a different sharing rule: **structure**, not the constant.



Operating regime

- $T \leq 5$ ms → **reservation** (KKT) wins
- $T = 10$–20 ms, short bursts → **back-pressure** wins
- Long bursts (FTP files, XR frames) → reservation at every T

Outline

1. Introduction and Motivation
2. Part A: IQ Compression by Matrix Decomposition
3. Part A2: Precoding Inversion and Kernel Programming
4. Part B: Sharing One Fronthaul Link
5. Part B: Results
6. Extended Study: Theory and Stronger Controllers
7. Conclusions and Future Work

What we do *not* claim

- **No real-world validation**: three simulated families. Family G uses standard 3GPP models but our own round-robin scheduler and single-link fronthaul.
- **Part A** compression figures are from the earlier run (the `src/` package is not in the repository); the inversion benchmark uses random well-conditioned matrices, not channel matrices.
- **Kernels**: 10–15$\times$ is vs. single-threaded numpy+LAPACK (2.5–4$\times$ per thread); no MKL/Eigen comparison; on CPU the Newton tracker loses to a tuned exact inverse; RMT-WS components are largely prior art.
- **No closed-loop optimality or stability claim**: propositions cover feasibility, equal weighted risk, backlog-first and a linear miscalibration bound.
- **No universal superiority**: only $-4\,\%$ vs. proportional under flash crowds; loses at $T = 20$ ms, at ≥ 0.86 load, near-tie at $\tau = 5$ ms; ML adds $\approx 2\,\%$ under shift.
- **Priority trade**: the claim holds for LL weight $\geq$ 2–3.
- In the extended comparison, CEM policy search and predictive MaxWeight beat the KKT rule in 9/10 scenarios.

Conclusions

Part A

Structure-aware encoders: **13–16**$\times$ at ≈ -20 dB (BFP $\sim 4\times$), **3–6**$\times$ **faster** than SVD. A learned *iteration* works where a direct neural inverse fails. A **fused C/AVX-512 RZF kernel** runs the whole precoder 10–15$\times$ faster than numpy+LAPACK, fitting $K \leq 16$ in a 0.5 ms slot; RU-side precoding cuts fronthaul by $\approx 72\,\%$.

Part B

Forecasting the **deadline-feasible rate** as a **range** and splitting the link with a closed-form **KKT rule** is the best of 12 controllers in all scenarios of two families: -11 **to** -43 **%** vs. the forecast median, -19 **to** -70 **%** vs. deadline-aware reactive, every seed, $p < 0.01$.

Extended study

r^* linked to effective bandwidth (bounds valid but loose). Reward-learned class levels and predictive back-pressure go further: the gap is **structural**. Short interval $\rightarrow$ reservation; long interval, short bursts $\rightarrow$ back-pressure.

Take-away

Compression sets the load regime; allocation matters most between $\approx$0.5 and 0.8 load. Keep the structure, learn only what is uncertain.

Future work

Part A

- Restore the `src/` package and regenerate Exp. 1–4
- Benchmark on realistic channel matrices
- SIMD / GPU real-time encode path ($< 100\,\mu$s)
- ACAFS as a near-RT RIC xApp (E2SM)
- Kernels: compare vs MKL batched / Eigen; AMX; GPU/NPU for the Newton path; cache-blocking for $K > 32$
- Benchmark vs ChebI-NS, Chebyshev-RZF, TPE

A + B

- Make the compression width a **per-cell knob** of the allocation rule (trade signal quality for load)

Part B

- Validation on measured per-cell PRB traces
- Controller that **switches** between reservation and back-pressure from estimated burst length vs. interval
- Coupled calibration (adjust λ, or calibrate on unconstrained error)
- Skill-weighted mix of learned and window quantiles as fallback
- Switch-level queue model; compiled implementation for an O-DU
- Stronger policy search (neural policy, more episodes)

Deliverables and reproducibility

Repository

- `matinv_bench/` — inversion benchmark, InverseNets
- `cmatinv/`, `experiments_v2/` — complex inversion, RMT-WS, 38.901 channels, Exp. 5–18
- `kernels/` — fused C/AVX-512/OpenMP precoder
- `experiments/` — Part A Exp. 1–4, extended pipeline
- `fhlm/` — simulator, traffic (S, G, M), forecasters, 19 controllers, theory, figures, tables
- `tests/` — 16 + 14 + 8 tests (capacity every slot, no future leakage, bound validity, . . .)
- `results/` — all CSVs / logs / figures

Papers NCC 2027 submission (6 pp.); 11-page extended manuscript.

Reproduce

```
python -m pytest tests -q
python -m fhlm.train \
  --out models/fhlm_quantile_gbm_T20_d2.pkl
python -m fhlm.run_experiments tune
python -m fhlm.run_experiments main
python -m fhlm.make_figures
bash experiments/run_extended.sh
sh kernels/build.sh
python experiments_v2/run_all.py
```

Every number in the papers is a macro generated from the CSVs.

Key references

- O-RAN WG4, *Control, User and Synchronization Plane Specification* (split 7-2x, BFP compression).
- 3GPP TR 38.801, TR 38.901, TS 38.214, TR 36.814, TR 38.838.
- L. M. P. Larsen, A. Checko, H. L. Christiansen, "A survey of the functional splits proposed for 5G mobile crosshaul networks," *IEEE COMST*, 2019.
- G. O. Pérez, J. A. Hernández, D. Larrabeiti, "Fronthaul network modeling and dimensioning meeting ultra-low latency requirements for 5G," *JOCN*, 2018.
- L. Wang, S. Zhou, "On the fronthaul statistical multiplexing gain," *IEEE Commun. Lett.*, 2017.
- S. Lagén et al., "Fronthaul compression control for shared fronthaul access networks," *IEEE Commun. Mag.*, 2022.
- D. Bega et al., "DeepCog" (2019) and "AZTEC" (2020): forecast-driven capacity allocation.
- I. Gibbs, E. Candès, "Adaptive conformal inference under distribution shift," NeurIPS 2021.
- N. Halko, P. G. Martinsson, J. A. Tropp, "Finding structure with randomness," *SIAM Review*, 2011.
- J.-Y. Le Boudec, P. Thiran, *Network Calculus*, Springer, 2001.
- M. J. Neely, *Stochastic Network Optimization*, 2010; L. Huang et al., predictive back-pressure, 2016.
- G. Hadley, T. M. Whitin, *Analysis of Inventory Systems* (multi-item newsvendor), 1963.
- W. Willinger et al., self-similar ON/OFF traffic, *IEEE/ACM ToN*, 1997.
- D. Anick, D. Mitra, M. M. Sondhi, Markov fluid sources, *BSTJ*, 1982.

Thank you

Questions?

Mayank Yadav ▪ Harshvardhan Choudhary

Backup: what is new here?

None of the ingredients is new on its own (quantile GBM, constrained newsvendor / KKT, water-filling, conformal inference, network-calculus service curves). The candidate contribution is the **combination and its fair evaluation** in the shared-fronthaul setting:

1. Forecasting the **deadline-feasible rate** r^*, not the traffic volume.
2. A rule that turns the forecast **range** into budgets under the shared-link constraint by cutting every cell at the same weighted risk — margins *derived*, not tuned.
3. An evaluation that is fair by construction and reports where the method does **not** win.

Closest work: Lagén et al. 2022 (shared 7-2x link, reactive compression, no forecast); Bega et al. DeepCog/AZTEC (single predicted number, no per-cell deadlines); Cohen et al. 2023 (conformal for one URLLC stream; our ablation shows per-stream correction does not carry over).

Backup: likely questions

Leakage? Train 1000–1015, val 2000–2003, test 3000–3019; forecaster frozen before test; features use past slots only (tested); normalisation by config constants.

Why GBM, not LSTM? Six quantile models train in 12 s, well calibrated, CPU-only; the ablation shows the model is not where most of the gain is.

Why synthetic traffic? No public per-cell PRB-level fronthaul traces (ColO-RAN is $\geq$250 ms); hence Family G built from 3GPP models.

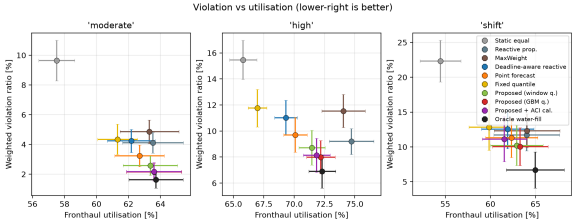
Why are violations high? Heavy-tailed bursts at 0.8 load with budgets held 10 ms; even the oracle loses 13 % of eMBB bits. Paired differences are the result.

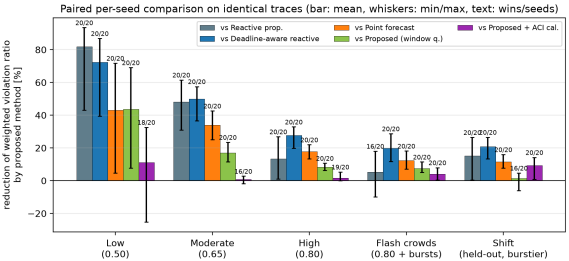
Is the allocation optimal? Only for the one-interval surrogate (separable concave, KKT = global optimum). No closed-loop claim.

Why does ACI hurt? LL misses in a coupled split are partly caused by the shortage split, not forecast bias; offsets over-reserve LL.

Could the DU export its schedule? Yes where an interface exists (e.g. CTI); our setting is budgets committed before the schedule is known.

Backup: trade-off and paired reductions





Backup: Family G figure and telemetry delay

